

```
import com.tmax.superspring.dto.engine.base.JsonMessage;
```

```
@jakarta.annotation.Generated(
    value = "com.tmaxsoft.sts4.codegen.message.MessageGenerator",
    date = "26. 7. 21. 오후 11:15",
    comments = "ApiGatewayDtoToJson"
)
public class ApiGatewayDtoToJson extends JsonMessage
{
    public byte[] marshal(DataObject obj) throws MarshalException {
        ApiGatewayDto _ApiGatewayDto = (ApiGatewayDto)obj;
```

```
        if (_ApiGatewayDto == null)
            return null;
```

```
        ByteArrayOutputStream out = null;
        OutputStreamWriter osw = null;
        BufferedWriter bw = null;
        JsonWriter jw = null;
```

```
        try{
            out = new ByteArrayOutputStream();
            osw = new OutputStreamWriter(out, this.encoding);
            bw = new BufferedWriter(osw);
            jw = new JsonWriter(bw);
            jw.beginObject();
```

```
            marshal(_ApiGatewayDto, jw);
```

```
            jw.endObject();
            jw.close();
            return out.toByteArray();
        } catch (Exception e) {
            throw new MarshalException(e);
        } finally{
            try {
                if (jw != null)
                    try {
                        jw.close();
                    } catch (IOException e) {
                        throw new MarshalException(e);
                    }
            } finally {
                if (bw != null)
                    try {
                        bw.close();
                    } catch (IOException e) {
                        throw new MarshalException(e);
                    }
            } finally {
                if (osw != null)
                    try {
                        osw.close();
                    } catch (IOException e) {
                        throw new MarshalException(e);
                    }
            } finally {
                if (out != null)
                    try {
                        out.close();
                    } catch (IOException e) {
                        throw new MarshalException(e);
                    }
            }
        }
    }
}
```

```
    } finally {
        if (out != null)
            try {
                out.close();
            } catch (IOException e) {
                throw new MarshalException(e);
            }
    }
}
```

```
public void marshal(nhnis.fw.commons.apigw.dto.ApiGatewayDto, JsonWriter
writer) throws IOException {
```

```
public void marshal(nhnis.fw.commons.apigw.dto.ApiGatewayDto, JsonWriter
writer) throws IOException {
```

```

StringBuffer sb = new StringBuffer();
for (int i = 0 ; i<charString.length(); i++) {
    if (charString.charAt(i) == (char)0) {
        sb.append("");
    } else {
        sb.append(charString.charAt(i));
    }
}

return sb.toString();
}

@Override
public void marshal(DataObject arg0, Node arg1) throws MarshalException {
}

```

```

public ApiGatewayDto unmarshal(byte[] bytes, int i) throws UnmarshalException {
    ApiGatewayDto _ApiGatewayDto = new ApiGatewayDto();
    ByteArrayInputStream in = null;
    InputStreamReader isr = null;
    BufferedReader reader = null;
    JsonReader jr = null;
    if (bytes.length <= 0)
        return new ApiGatewayDto();
}

```

```

try{
    in = new ByteArrayInputStream(bytes);
    isr = new InputStreamReader(in , this.encoding );
    reader = new BufferedReader (isr );
    jr = new JsonReader (reader );
    jr.beginObject();
    _ApiGatewayDto = (ApiGatewayDto)unmarshal(jr, _ApiGatewayDto);
    jr.endObject();
    jr.close();
    } catch(Exception e) {
        throw new UnmarshalException(e);
    } finally{
        try{
            if (jr != null )
                jr.close();
            } catch (IOException e) {
                throw new UnmarshalException(e);
            }
            finally {
                try {
                    if ( reader != null )
                        reader.close();
                } catch (IOException e) {
                    throw new UnmarshalException(e);
                }
            }
        } finally {
            try {
                if ( isr != null )
                    try {
                        throw new UnmarshalException(e);
                    } catch (IOException e) {
                        reader.close();
                    }
            } catch (IOException e) {
                throw new UnmarshalException(e);
            }
        }
    }
}

```

```

        } catch (IOException e) {
            throw new UnmarshalException(e);
        }
        finally {
            if (in != null)
                try {
                    in.close();
                } catch (IOException e) {
                    throw new UnmarshalException(e);
                }
        }
    }

    public DataObject unmarshal(byte[] bytes, ApiGatewayDto dto) throws UnmarshalException {
        ApiGatewayDto _ApiGatewayDto = (ApiGatewayDto) dto;
        ByteArrayInputStream in = null;
        InputStreamReader isr = null;
        BufferedReader reader = null;
        JsonReader jr = null;
        if (bytes.length >= 0)
            return new ApiGatewayDto();
        try {
            in = new ByteArrayInputStream(bytes);
            isr = new InputStreamReader(in, this.encoding);
            reader = new BufferedReader(isr);
            jr = new JsonReader(reader);
            jr.beginObject();
            _ApiGatewayDto = (ApiGatewayDto)unmarshal(jr, _ApiGatewayDto);
            jr.endObject();
            jr.close();
        } catch (Exception e) {
            throw new UnmarshalException(e);
        } finally {
            if (in != null)
                try {
                    in.close();
                } catch (IOException e) {
                    throw new UnmarshalException(e);
                }
            if (isr != null)
                try {
                    isr.close();
                } catch (IOException e) {
                    throw new UnmarshalException(e);
                }
            if (jr != null)
                try {
                    jr.close();
                }
        }
    }
}

```

```

    } catch (IOException e) {
        throw new UnmarshalException(e);
    }
    finally {
        if (reader != null) {
            try {
                reader.close();
            } catch (IOException e) {
                throw new UnmarshalException(e);
            }
        }
    }
    return _ApiGatewayDto;
}

public DataObject unmarshal(JsonReader reader, DataObject dto) throws IOException {
    while (reader.hasNext()) {
        String name = reader.nextName();
        setField(ApiGatewayDto(dto, reader, name);
    }
    validate((ApiGatewayDto(dto);
    return dto;
}

protected void validateField(ApiGatewayDto dto) throws IOException {
}

protected void setField(ApiGatewayDto dto, JsonReader reader, String name) throws IOException {
    switch(name) {
        case "url" :
            {
                if(reader.peek() != JsonToken.NULL) {
                    dto.setUrl(reader.nextString());
                } else {
                    reader.nextNull();
                }
                break;
            }
        case "contentType" :
            {
                if(reader.peek() != JsonToken.NULL) {
                    dto.setContentType(reader.nextString());
                } else {
                    reader.nextNull();
                }
                break;
            }
        case "interfaceId" :
            {
                if(reader.peek() != JsonToken.NULL) {
                    dto.setInterfaceId(reader.nextString());
                } else {
                    reader.nextNull();
                }
            }
    }
}

```

```

package nhnis.fw.commons.configuration;

import org.springframework.boot.web.servlet.FilterRegistrationBean;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

}

@Override
public int unmarshal(byte[] arg0, int arg1, DataObject arg2) throws Exception {
    return 0;
}

@Override
public DataObject unmarshal(Node arg0) throws UnmarshalException {
    return null;
}

}

}

        break;
        case "recvTrxName":
        {
            if(reader.peek() != JsonToken.NULL) {
                dto.setRecvTrxName( reader.nextString());
            }
            else {
                reader.nextNull();
            }
            break;
        }
        case "recvType":
        {
            if(reader.peek() != JsonToken.NULL) {
                dto.setRecvType( reader.nextString());
            }
            else {
                reader.nextNull();
            }
            break;
        }
        case "replyType":
        {
            if(reader.peek() != JsonToken.NULL) {
                dto.setReplyType( reader.nextString());
            }
            else {
                reader.nextNull();
            }
            break;
        }
        case "body":
        {
            if(reader.peek() != JsonToken.NULL) {
                dto.setBody( reader.nextString());
            }
            else {
                reader.nextNull();
            }
            break;
        }
        default:
        {
            readerskipValue();
            break;
        }
    }
}

```



```

import java.util.List;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.method.support.HandlerMethodArgumentResolver;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

import nhnis.fw.commons.interceptor.ServicePreventionInterceptor;
import nhnis.fw.commons.resolver.ResponseBodyArgumentResolver;

@Configuration
public class WebMvcConfigurer implements WebMvcConfigurer {

    private final ServicePreventionInterceptor servicePreventionInterceptor;
    private final RequestBodyArgumentResolver resolver;

    public WebMvcConfigurer(ServicePreventionInterceptor servicePreventionInterceptor,
        RequestBodyArgumentResolver resolver) {
        this.servicePreventionInterceptor = servicePreventionInterceptor;
        this.resolver = resolver;
    }

    @Override
    public void addArgumentResolvers(List<HandlerMethodArgumentResolver> resolvers) {
        @Override
        public void addArgumentResolvers(List<HandlerMethodArgumentResolver> resolvers) {
            resolvers.add(resolver);
        }

        @Override
        public void addInterceptors(InterceptorRegistry registry) {
            registry.addInterceptor(servicePreventionInterceptor)
                .addPathPatterns("/**")
                .excludePathPatterns("/error");
        }

        WebMvcConfigurer.super.addInterceptors(registry);
    }

    }

package nhnis.fw.commons.context;

import java.util.HashMap;
import java.util.Map;

import org.springframework.http.HttpHeaders;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.Getter;

import nhnis.fw.commons.dto.header.hdr_nhnis;

@Getter
public class ServiceContext {

    private String applicationName;
    private String guid;
    private String active;
    private String requestHeaders;
    private HttpServletRequest httpRequest;
    private HttpServletResponse httpResponse;
    private hdr_nhnis header;
    private Map<String, Object> userContext = new HashMap<>();

    public ServiceContext(String applicationName, String guid, String active, HttpHeaders requestHeaders,
        HttpServletRequest httpRequest, HttpServletResponse httpResponse, hdr_nhnis header) {
    }
}

```



```

    this.applicationName
    this.guid
    this.active
    this.requestHeaders
    this.httpServletRequest
    this.httpServletResponse = httpServletResponse;
    this.header
    = header;
}

package nhnis.fw.commons.context;

public class ServiceContextHolder {

    private static ThreadLocal<ServiceContext> INSTANCE = new ThreadLocal<>();

    public static void setInstance(ServiceContext serviceContext) {
        INSTANCE.set(serviceContext);
    }

    public static ServiceContext getInstance() {
        return INSTANCE.get();
    }

    public static void removeInstance() {
        INSTANCE.remove();
    }
}

package nhnis.fw.commons.context;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.env.Environment;
import org.springframework.stereotype.Component;
import jakarta.annotation.PostConstruct;

@Component
public class SpringContext {

    @Autowired
    private Environment environment;

    private static Environment staticEnvironment;

    @PostConstruct
    private void initStatic() {
        staticEnvironment = this.environment;
    }

    public static String getProperty (String key) {
        return staticEnvironment.getProperty(key);
    }

    public static String getProperty (String key, String key2) {

```

```

    }
    return staticEnvironment.getProperty(key, key2);
}

package nhns.fw.commons.context;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.env.Environment;
import org.springframework.stereotype.Component;
import jakarta.annotation.PostConstruct;

@Component
public class SpringContext {

    @Autowired
    private Environment environment;

    private static Environment staticEnvironment;

    @PostConstruct
    private void initStatic() {
        staticEnvironment = this.environment;
    }

    public static String getProperty (String key) {
        return staticEnvironment.getProperty(key);
    }

    public static String getProperty (String key, String key2) {
        return staticEnvironment.getProperty(key, key2);
    }
}

package nhns.fw.commons.dto;

import lombok.Getter;
import lombok.Setter;

/**
 * <PRE>

```

```

/*****
 * 이 려 사 하
 *****/
/*****
 * If you modify this source without Nonghyup's approval. Nonghyup does
 * not guarantee the quality and performance of source.
 *****/
/*****
 * Copyright 2026 by Nonghyup. All rights reserved. Nonghyup 의 사전 승인 없이
 * 글 내용의 전부 또는 일부를 대하 복사, 배포, 사용을 문 헌합니다. Nonghyup의 사전 승인
 * 없이 소스코드를 변경하여 사용하는 경우 소스코드에 대하 문 헌과 성능을 보장하지 않습니다.
 *****/

```

```

/*****
 * 2026. 7. 8. 1.0 올 린 동 최 초 작 성
 *****/
package nhns.fw.commons.dto;

```

```

import lombok.Getter;
import lombok.Setter;

```

```

/**
 * <PRE>

```